

WLD

N1MM Field Day Tracker

Volledige handleiding & technische documentatie

Sectie WLD — ON6WL · versie blok C · {stamp}

1. Wat is deze applicatie?

De **N1MM Field Day Tracker** is een klein programma dat *naast* N1MM Logger+ draait tijdens een radiovelddag. N1MM blijft de officiële logger; de tracker toont in een **matrix** welke deelnemende stations op welke banden al gewerkt zijn, met live-updates, en publiceert optioneel een alleen-lezen webpagina zodat thuisblijvers en andere secties kunnen meevolgen. Hij vervangt de vroegere handmatige Excel.

Kernidee: N1MM stuurt bij elk gelogd QSO een berichtje over het netwerk (UDP). De tracker vangt die berichtjes op, herkent of het om een deelnemend station gaat, en kleurt de juiste cel in de matrix. Alles gebeurt lokaal op de laptop; internet is enkel nodig als je de publieke pagina wil publiceren.

Belangrijk onderscheid: de tracker **sniff't niet**. Hij zet geen netwerkkaart in "afluistermodus" en vangt geen verkeer op dat niet voor hem bedoeld is. Hij maakt gewoon een eigen "brievenbus" (een UDP-poort) open en leest enkel wat N1MM daar bewust naartoe stuurt. Zie hoofdstuk 8.

2. Snel van start (functioneel)

2.1 Installeren en starten

De volledige klik-voor-klik-instructies staan in het aparte **DRAAIBOEK**. Samengevat: Python installeren, het project uitpakken, een virtuele omgeving maken, de pakketten installeren en starten met `python -m app.main`. De browser opent dan vanzelf op `http://127.0.0.1:8765`.

2.2 De deelnemerslijst inladen

Importeer je Excel of CSV via **Manage -> deelnemerslijst**, of bij het starten met `--import-excel`. De lijst wordt herkend op **kolomkoppen** (eerste rij), niet op volgorde. Herkende koppen: Callsign (Call/Callsign/Roepnaam, verplicht), Naam, Club, Categorie, Sectie, Opmerking, en bandkolommen zoals 40m/80m/160m. Extra kolommen worden genegeerd.

2.3 De zes weergaven

Onderaan de kop kies je tussen: **Matrix** (stations x banden), **Nog te werken**, **Per band**, **Per station**, **Per bron-PC** (welke N1MM-computer wat stuurde, met live/stale-indicatie) en **Statistiek**. Filteren kan op zoekterm, status, band, categorie en sectie; een knop **Clear filters** herstelt alles.

2.4 Handmatige statussen (overrides)

Tik op een cel en kies **Mark worked**, **Mark NOT worked** of **Exclude**, eventueel met reden (bv. "papieren log"). Een handmatige status **wint altijd** van wat N1MM zegt en is herkenbaar aan het (pen)-symbool. **Clear manual status** herstelt de automatische status.

3. Instellingen — wat moet waar?

3.1 In N1MM

Open in N1MM: **Config > Configure Ports, Mode Control, Audio, Other...** > tabblad **Broadcast Data**. Vink **Contacts** aan (*niet* Lookup). Zet de bestemming op het adres + de poort van de laptop waarop de tracker draait:

- 127.0.0.1:12060 als N1MM op **dezelfde** laptop draait als de tracker;
- het netwerk-IP van de trackerlaptop (bv. 192.168.1.50:12060) als N1MM op een **andere** pc draait — zet dan in de tracker het luisteradres op 0.0.0.0.

Kies als contesttype **FDREG1**. Log een test-QSO; de cel hoort binnen 5 seconden groen te kleuren.

3.2 In de tracker (Manage -> Settings)

Bij de technische velden staan blauwe (?)**-cirkels** met directe uitleg. De belangrijkste instellingen:

- **UDP listen address / port**: waar de tracker luistert. Poort moet exact overeenkomen met N1MM. Na wijzigen herstart de ontvanger vanzelf.
- **Strict callsign matching**: UIT (aanbevolen) = ON4BAF/P en ON4BAF zijn hetzelfde station; AAN = enkel de exacte roepnaam telt.
- **Stale after**: na hoeveel seconden stilte een bron-PC als STALE (mogelijk kabel/netwerkprobleem) getoond wordt.
- **Statuskleuren** en **exportmap** naar keuze.

3.3 Windows Firewall

De eerste keer vraagt Windows mogelijk om netwerktoegang. Klik **Toegang toestaan** — dat is nodig om de N1MM-gegevens te ontvangen.

4. Velddagen beheren

Er is altijd precies één **actieve** velddag. **N1MM schrijft altijd naar de actieve velddag**. Je kan velddagen aanmaken (optioneel gekopieerd van een bestaande), bewerken, afsluiten en heropenen. Een **afgesloten** (CLOSED) velddag negeert binnenkomende QSO's; bekijken en exporteren blijft.

4.1 Exporteren, importeren, verwijderen

Met de **(export)-knop** exporteer je een volledige velddag (instellingen, deelnemers, alle QSO's en manuele statussen) naar één .fdttracker-bestand. Op een andere pc importeer je dat om verder te werken — importeren maakt altijd een *nieuwe* velddag en overschrijft nooit iets. De **(prullenbak)-knop** (enkel bij niet-actieve velddagen) verwijdert een velddag definitief; ter beveiliging moet je het woord **DELETE** typen.

4.2 Inhalen na uitval

Stond de tracker een tijd uit terwijl in N1MM werd doorgelogd? UDP draagt enkel live-verkeer, dus die QSO's zijn niet vanzelf ontvangen. Haal ze in via **N1MM -> File -> Export -> Export to ADIF** en in de tracker **Manage -> ADIF import**. Reeds bekende QSO's worden overgeslagen (dedup), dus dit kan zo vaak als nodig zonder dubbels.

5. Publiceren naar het publiek (GitHub Pages)

Eenmalige opzet: maak een publieke repo (bv. `velddag-live`), zet GitHub Pages aan, maak een fine-grained token met **Contents: Read and write** op enkel die repo, en koppel dat in **Manage -> Publish**. Vul de repository in als `owner/naam` (niet de volledige https-link). Daarna volstaat **Publish now**, of zet automatisch publiceren aan.

De publieke pagina past zich automatisch aan de velddagstatus aan:

Situatie	Wat het publiek ziet
Actieve velddag, binnen de periode	De live matrix
Vóór de start	Aankondiging met datum en aftelling
Geen actieve velddag	"Geen actieve velddag" + eventueel de volgende geplande datum
Vanaf 1 week na het einde	Resultaten worden niet meer getoond

Bij de niet-live toestanden staat er ook géén deelnemers- of QSO-data in het gepubliceerde bestand. Zonder internet blijft de tracker gewoon werken; het publiceren pauzeert dan netjes en hervat vanzelf.

6. Wat als de app crasht?

Elk QSO wordt bij binnenkomst onmiddellijk en atomisch op schijf bewaard — zelfs een stroomuitval midden in een schrijfactie kan geen half bestand achterlaten. Herstarten duurt minder dan een seconde en **alle** QSO's, manuele statussen en instellingen staan er terug. Als laatste vangnet staat elk ruw pakket in `raw_packets.log`.

7. Technische opbouw — overzicht

De applicatie is één Python-proces met drie draden (threads): een **UDP-luisteraar**, een **HTTP-webserver** en de **hoofddraad**. Alle gegevens staan in gewone JSON-bestanden. De rekenlogica (de "engine") is puur en bevat geen in-/uitvoer, waardoor ze volledig testbaar is zonder N1MM en zonder browser.

7.1 Mappenstructuur van de code

```
app/  
  config.py           platformdetectie, paden, poorten  
  core/  
    models.py        FieldDay, Station, QSO, Override, ...  
    status.py        de 5 celstatussen + prioriteit  
    callsign.py      normalisatie ON4BAF/P == ON4BAF  
    band_plan.py     band uit frequentie (IARU R1)  
    matching.py      koppelt QSO aan deelnemer+band  
    sync_engine.py   bouwt en onderhoudt de matrix  
  ingest/  
    n1mm_listener.py >>> de UDP-luisteraar (zie hfdst 8)  
    n1mm_parser.py   ontleedt N1MM's XML-berichten  
    adif_importer.py vangnet: volledig log inlezen  
    station_importer.py Excel/CSV-deelnemerslijst  
  storage/  
    json_store.py    veilig schrijven (.tmp -> replace)  
    fieldday_repository.py per velddag een map  
    app_settings.py  globale instellingen  
  view/  
    snapshot_builder.py bouwt snapshot.json  
    static/  
      index.html, app.js, style.css  
  export/  
    csv_exporter.py, pdf_exporter.py  
  publish/  
    github_publisher.py, credentials.py  
  server.py          AppState + HTTP-server + JSON-API  
  main.py            opstart / CLI
```

7.2 De levensloop van één QSO

1. N1MM stuurt een UDP-pakket (XML) naar de trackerpoort.
2. De **listener** ontvangt het, schrijft het ruw weg en laat de **parser** het ontleden (band uit de frequentie).
3. De **sync-engine** zoekt of de roepnaam een deelnemer is en op welke band; de juiste cel wordt herrekend (een override wint altijd).
4. Het resultaat gaat atomisch naar schijf.
5. De browser haalt elke 5 seconden `snapshot.json` op en kleurt de cel — zonder handmatig verversen.

8. Hoe werkt het luisteren op de UDP-poort?

Dit is het hart van de tracker, dus hier in detail. De volledige logica zit in `app/ingest/nlmm_listener.py`.

8.1 Wat is UDP, en waarom?

UDP is een eenvoudig netwerkprotocol waarbij een zender losse "datagrammen" (pakketjes) naar een adres+poort stuurt, zonder verbinding op te bouwen en zonder bevestiging. N1MM gebruikt het om bij elk gelogd contact een berichtje te "broadcasten". Het past perfect: als er even een pakket verloren gaat, is dat niet erg — de volgende volledige sync of een ADIF-import herstelt de stand.

8.2 De brievenbus openen (bind)

De tracker maakt een UDP-socket en **bindt** die aan een adres en poort. Vanaf dat moment ontvangt hij enkel de pakketten die expliciet naar dat adres+poort gestuurd worden. Dit is de kern:

```
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM) # UDP
sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
sock.bind((self.host, self.port)) # 127.0.0.1 of 0.0.0.0, poort 12060
sock.settimeout(_SOCKET_TIMEOUT_S) # 0.5 s, zodat afsluiten netjes kan
```

- `SOCK_DGRAM` = UDP (in tegenstelling tot `SOCK_STREAM` = TCP).
- `bind()` claimt de poort. **127.0.0.1** = enkel deze laptop; **0.0.0.0** = ook andere pc's op het netwerk mogen sturen. Dit is géén sniffen: de andere pc moet nog altijd expliciet naar dit adres adresseren.
- `SO_REUSEADDR` laat toe de poort meteen opnieuw te gebruiken na een herstart, zonder "address already in use".
- De **timeout** van een halve seconde zorgt dat de ontvangstlus regelmatig "wakker wordt" om te kijken of er een stopsignaal is — zo sluit de app netjes af in plaats van vast te hangen op een blokkerende ontvangst.

Mislukt het binden (bv. omdat een andere plugin de poort al gebruikt), dan wordt dat gemeld en stopt de app **niet** — een gedocumenteerd veldprobleem dat nooit tot een crash mag leiden.

8.3 De ontvangstlus in een eigen draad

Het ontvangen gebeurt in een aparte **daemon-thread**, zodat de webserver en de rest van de app ondertussen gewoon doorlopen. De lus is bewust kogelvrij:

```
def _run(self):
    while not self._stop_event.is_set():
        try:
            data, address = self._socket.recvfrom(BUFFER_SIZE) # wacht op pakket
        except socket.timeout:
            continue # niets binnen 0.5 s: opnieuw kijken
        except OSError:
            break # socket gesloten tijdens afsluiten
        try:
            self._handle_datagram(data, address)
        except Exception:
            self.stats.errors += 1 # EEN slecht pakket mag NOOIT
            logger.exception(...) # de listener stoppen
```

Twee ontwerpregels springen eruit. Ten eerste: een fout bij één pakket wordt geteld en gelogd, maar stopt de listener nooit — tijdens een veldtag mag één vreemd bericht de hele opvolging niet platleggen. Ten tweede: de `recvfrom` met timeout maakt een **nette** stop mogelijk (het stopsignaal wordt elke halve seconde gecontroleerd).

8.4 Per pakket: tellen, versheid, doorgeven

Voor elk datagram bepaalt de listener uit welke **bron-PC** het komt (uit het bericht zelf, en anders het afzender-IP), werkt hij de tellers en de "laatst gezien"-tijd van die bron bij, en geeft hij het ontlede pakket door aan een **handler-callback**. Die callback (geleverd door de server) voedt de sync-engine en bewaart het ruwe bericht. De listener zelf bevat dus **geen** veldtaglogica — hij is puur transport. Dat scheidt "netwerk" van "betekenis" en maakt beide los testbaar.

8.5 Versheid per bron-PC (het handigste velddiagnose-middel)

Voor elke bron houdt de listener bij wanneer het laatste pakket binnenkwam. De weergave "Per bron-PC" toont zo of een N1MM-computer **LIVE** of **STALE** is. Valt er ergens een netwerkkabel uit, dan zie je die post binnen enkele seconden op STALE springen — vaak nog vóór de operator het zelf doorheeft.

```
def sources_status(self, threshold_seconds):
    now = utc_now()
    for source in self.sources.values():
        stale = (now - source.last_seen_utc).total_seconds() > threshold_seconds
        ... -> {name, last_seen, packet_count, stale}
```

8.6 De parser: welke pakketten tellen mee?

N1MM stuurt verschillende soorten berichten. De parser kijkt naar de **roottag** van de XML: `contactinfo` (nieuw QSO), `contactreplace` (gewijzigd) en `contactdelete` (verwijderd) tellen mee; andere zoals `lookupinfo` worden bewust **genegeerd** (dat is enkel een opzoeking, geen gelogd contact). De band wordt uit de frequentie afgeleid volgens het IARU R1-bandplan, zodat de tracker niet afhankelijk is van hoe N1MM de band benoemt.

9. De sync-engine en de matrix

De engine bepaalt per cel (station x band) de status uit vijf mogelijkheden, met een vaste prioriteit: een **override** (handmatig) wint altijd van een automatische status. De normalisatie van roepnamen wordt bij het matchen op beide kanten toegepast, telkens vanaf de originele roepnaam — daarom werkt de strict/loose-schakelaar ook met terugwerkende kracht na een hersync.

Een belangrijke garantie (geteste eigenschap): een **incrementele** update (één QSO erbij) geeft exact dezelfde matrix als een **volledige** hersync. Daardoor kan de tracker snel live bijwerken én is een volledige herberekening altijd veilig.

10. Opslag: veilig schrijven

Elke schrijfactie verloopt atomisch: eerst naar een `.tmp`-bestand, dat teruglezen en controleren, en pas dan met één `os.replace` op zijn plaats zetten. Een corrupt bestand wordt niet gewist maar opzijgezet als `.corrupt.<tijd>`; de app start dan verder met een lege structuur. Elke velddag heeft zijn eigen map met `fieldday.json`, `stations.json`, `received_qsos.json`, `overrides.json` en `raw_packets.log`.

11. Publiceren: hoe de webpagina in GitHub geraakt

Bij publiceren berekent de tracker eerst de **publicatiestatus** (live / aankomend / geen / verlopen) en bouwt daar de gepaste `snapshot.json` voor. Die snapshot en de webpagina (`index.html`, `app.js`, `style.css`) gaan via de GitHub Contents-API naar de repo; ongewijzigde bestanden worden overgeslagen (vergelijking op git-blob-sha). Het token staat in de OS-sleutelbos, nooit in een bestand. Bij geen internet stopt het publiceren meteen (geen herhaalstorm) en hervat het vanzelf.

12. Testen

De applicatie heeft ruim 360 geautomatiseerde tests (`python -m pytest tests\`): unit-tests voor callsign, bandplan, matching, engine en opslag; integratietests die een échte UDP-poort en HTTP-server gebruiken; en de regressietest dat incrementeel gelijk is aan een volledige hersync. Daarnaast is er een handmatig **TESTPLAN** met genummerde controles, met en zonder N1MM.

N1MM Field Day Tracker — WLD/ON6WL — 2026-07-22. Dit document hoort bij de handleiding (HANDLEIDING.md), het testplan (TESTPLAN.md), de architectuur (ARCHITECTUUR.md) en het draaiboek (DRAAIBOEK.md).